```
"book".rjust(15)       # => "           book"
"book".center(15, "0") # => "00000book000000"
```

If you want to change the case of a string, Ruby provides

- upcase, which puts all the characters into uppercase.
- downcase, which puts them all into lowercase.
- capitalize, which capitalizes the first letter of every word.
- swapcase, which swaps the case of every character in the string.

By default these methods all use Unicode case mapping, but all three take optional arguments for :ascii (which limits case behavior to only ASCII a-z characters) or :turkic (which adapts to the unique case mapping of Turkic languages). downcast, and downcast alone, takes :fold (which uses Unix case folding instead of case mapping).

Using Strings and Math-Like Operators

Strings do respond to some other binary operators. A string plus a string is a new string concatenating the two strings. Ruby will also concatenate strings with <<. Ruby will even concatenate strings if you put two strings next to each other with no operator between them:

```
"ab" + "cd"  # => "abcd"
"ab" << "cd" # => "abcd"
"ab" "cd"    # => "abcd"
```

The inverse of concatenation is prepending, if you need to for some reason Ruby provides prepend, as in "ab".prepend("cd").

A string can be multiplied by a non-negative integer to produce a new string that is the string that many times:

```
"ab" * 10 # => "abababababababababab"
```

Strings respond to the <=> and use Comparable, so all your comparison operators work. The <=> comparison is case-sensitive. A case-insensitive version is casecmp. The expression a.casecmp(b) is equivalent to a.downcase <=> b.downcase. The method casecmp? returns true if the downcased versions of each string are equal.

We've seen =~ and !~ for regular expression match and not-regular expression match. Strings also define === to return true on a regular expression match.

Finally, you can reverse a string with reverse, which returns a new string, or reverse! which changes the receiving string in place.

Encoding

The encoding determines how Ruby converts the bytes making up a string into individual characters. We can verify that Ruby correctly interprets π as a single character:

```
# encoding: utf-8
pi = "π"
puts "The size of a string containing π is #{pi.size}"
```

produces:

```
The size of a string containing π is 1
```